CODIGO FUENTE

Aplicación móvil con Machine Learning para el Diagnóstico preliminar de Dengue en un Puesto de Salud

CONTENIDO GENERAL

A. PANTALLA DE LOGIN Y REGISTRO 3
Pantalla de inicio de sesión 3
Pantalla de registrar usuario 3
B. USUARIOS CON ROL “PACIENTE” 5
Pantalla de Realizar diagnóstico 5
Pantalla historial de diagnósticos 7
Pantalla de medidas preventivas 9
Pantalla de reportes de diagnósticos 10
C. USUARIOS CON ROL “MÉDICO” 13
Pantalla listado de pacientes asignados 13
Pantalla de Validación de diagnósticos 15
Pantalla de Reportes de Pacientes 18
D. USUARIOS CON ROL “ADMINISTRADOR” 23
Pantalla de Administradores y Médicos 23
Pantalla de Listado de pacientes 25
Pantalla de Medidas Preventivas 26

A. PANTALLA DE LOGIN Y REGISTRO

Pantalla de inicio de sesión

```python
def login(request):

    data = request.data or {}
    correo = str(data.get("correo") or "").strip().lower()
    contrasena_plana = str(data.get("contrasena") or "")

    if not correo or not contrasena_plana:
        return Response(
            {"status": "error", "message": "Ingrese correo y contraseña."},
            status=status.HTTP_400_BAD_REQUEST,
        )

    try:
        usuario = Usuario.objects.get(correo__iexact=correo)
    except Usuario.DoesNotExist:
        return Response(
            {"status": "error", "message": "Credenciales inválidas."},
            status=status.HTTP_401_UNAUTHORIZED,
        )

    if not check_password(contrasena_plana, usuario.contrasena):
        return Response(
            {"status": "error", "message": "Credenciales inválidas."},
            status=status.HTTP_401_UNAUTHORIZED,
        )

    try:
        session_token = secrets.token_urlsafe(48)
        usuario.verification_token = session_token
        usuario.token_expiration = None
        usuario.save(update_fields=["verification_token", "token_expiration"])

        persona = Persona.objects.select_related("id_rol").get(id_persona=usuario.id_persona_id)
        persona_serializer = PersonaSerializer(persona)

        return Response(
            {
                "status": "success",
                "message": "Logir exitoso.",
                "token_type": "Bearer",
                "access_token": session_token,
                "usuario": persona_serializer.data,
            },
            status=status.HTTP_200_OK,
        )
    except Exception as e:
        return Response(
            {"status": "error", "message": f"Error interno: {str(e)}"},
            status=status.HTTP_500_INTERNAL_SERVER_ERROR,
        )
```

Descripción: se muestra el código para el botón Iniciar Sesión**.** Se crea una función para ingresar al sistema mediante un correo y contraseña, validando también el rol del usuario

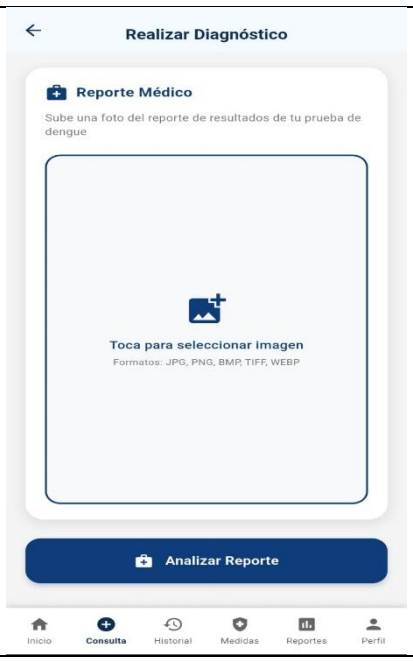
Pantalla de registrar usuario

```python
def register(request):
    data = request.data or {}

    try:
        norm = _validar_campos_register(data)
    except ValueError as e:
        return Response({"status": "error", "message": str(e)}, status=status.HTTP_400_BAD_REQUEST)

    try:
        rol_paciente = Rol.objects.get(nombre_rol__iexact="PACIENTE")
    except Rol.DoesNotExist:
        return Response(
            {"status": "error", "message": "No existe el rol PACIENTE en la base de datos."},
            status=status.HTTP_500_INTERNAL_SERVER_ERROR
        )

    if Usuario.objects.filter(correo__iexact=norm["correo"]).exists():
        return Response({"status": "error", "message": "El correo ya está registrado."}, status=status.HTTP_409_CONFLICT)

    if Persona.objects.filter(dni=norm["dni"]).exists():
        return Response({"status": "error", "message": "El DNI ya está registrado."}, status=status.HTTP_409_CONFLICT)

    try:
        with transaction.atomic():
            persona = Persona.objects.create(
                id_persona=uuid.uuid4(),
                nombres=norm["nombres"],
                apellidos=norm["apellidos"],
                dni=norm["dni"],
                sexo=norm["sexo"],
                fecha_nacimiento=norm["fecha_nacimiento"],
                celular=norm["celular"],
                id_rol=rol_paciente
            )

            usuario = Usuario.objects.create(
                id_usuario=uuid.uuid4(),
                id_persona=persona,
                correo=norm["correo"],
                contrasena=make_password(norm["contrasena"]),
            )

            return Response(
                {
                    "status": "success",
                    "message": "Cuenta de PACIENTE creada exitosamente.",
                    "id_persona": str(persona.id_persona),
                    "id_usuario": str(usuario.id_usuario),
                },
                status=status.HTTP_201_CREATED,
            )
    except Exception as e:
        return Response(
            {"status": "error", "message": f"No se pudo registrar: {str(e)}"},
            status=status.HTTP_500_INTERNAL_SERVER_ERROR
```

Descripción: se muestra el código para el botón **Registrarse.** Se crea una función para registrar un paciente con los campos que se muestran en las imágenes.

B. USUARIOS CON ROL “PACIENTE”

Pantalla de Realizar diagnóstico



```python
def predecir_dengue(request):
    if 'imagen' not in request.FILES:
        return Response(
            {
                'status': 'error',
                'error': 'Se requiere una imagen para el diagnóstico'
            },
            status=status.HTTP_200_OK
        )

    try:
        imagen_file = request.FILES['imagen']

        # Validar extensión
        extensiones_permitidas = ['.jpg', '.jpeg',
                                  '.png', '.bmp', '.tiff', '.webp']
        file_name = imagen_file.name.lower()
        if not any(file_name.endswith(ext) for ext in extensiones_permitidas):
            return Response(
                {
                    'status': 'error',
                    'error': f'Tipo de archivo no permitido. Use: {", ".join(extensiones_permitidas)}'
                },
                status=status.HTTP_200_OK
            )

        inicio_tiempo = time.time()

        # 1 SUBIR IMAGEN A SUPABASE STORAGE
        try:
            # Crear prefijo con ID de usuario
            user_id_str = str(request.usuario.id_usuario)

            # Subir imagen a Supabase
            object_path, public_url = _supabase_upload_image(
                file_obj=imagen_file,
                path_prefix=user_id_str,
                root_folder="diagnosticos",
                filename_hint=imagen_file.name
            )

            # Guardar URL pública en la base de datos
            url_imagen = public_url

        except Exception as upload_error:
            return Response(
                {
                    'status': 'error',
                    'error': f'Error al subir imagen a storage: {str(upload_error)}'
                },
                status=status.HTTP_200_OK
            )
```

```python
        temp_path = f"temp_{uuid.uuid4()}_{imagen_file.name}"
        imagen_file.seek(0)  # Resetear cursor después de leer en upload
        temp_file_path = default_storage.save(
            temp_path, ContentFile(imagen_file.read()))
        temp_full_path = default_storage.path(temp_file_path)

        try:
            # Analizar imagen con Gemini
            resultado_analisis = dengue_service.analizar_imagen_dengue(
                temp_full_path)

            # 1) Imagen no es un reporte válido
            if resultado_analisis == "FORMATO_INCORRECTO":
                return Response(
                    {
                        'status': 'error',
                        'error': 'La imagen no parece ser un reporte de resultados de dengue válido'
                    },
                    status=status.HTTP_200_OK
                )
            elif resultado_analisis is None:
                return Response(
                    {
                        'status': 'error',
                        'error': 'Error al procesar la imagen. Intente nuevamente.'
                    },
                    status=status.HTTP_200_OK
                )

            # 2) Campos incompletos detectados
            if isinstance(resultado_analisis, dict) and str(resultado_analisis.get('status', '')).upper() == 'CAMPOS_INCOMPLETOS':
                return Response(
                    {
                        'status': 'CAMPOS_INCOMPLETOS',
                        'mensaje': resultado_analisis.get(
                            'mensaje',
                            'Faltan campos obligatorios en el reporte de dengue.'
                        ),
                        'campos_faltantes': resultado_analisis.get('campos_faltantes', []),
                    },
                    status=status.HTTP_200_OK
                )

            # 3) Datos ya normalizados desde dengue_model
            try:
                edad_val = int(resultado_analisis.get('edad'))
            except (TypeError, ValueError):
                raise ValueError('Edad no válida en los datos extraídos.')

            sexo_raw = str(resultado_analisis.get(
                'sexo', '') or '').strip().upper()
            sexo_interno = 'M' if sexo_raw.startswith('M') else 'F'
```

```python
            try:
                ns1_val = int(resultado_analisis.get('ns1'))
                igm_val = int(resultado_analisis.get('igm'))
                igg_val = int(resultado_analisis.get('igg'))
                plaquetas_val = int(resultado_analisis.get('plaquetas'))
                hematocrito_val = float(resultado_analisis.get('hematocrito'))
            except (TypeError, ValueError):
                raise ValueError(
                    'Valores numéricos inválidos en los datos extraídos.')

            datos_paciente = {
                'edad': edad_val,
                'sexo': sexo_interno,
                'ns1': ns1_val,
                'igm': igm_val,
                'igg': igg_val,
                'plaquetas': plaquetas_val,
                'hematocrito': hematocrito_val,
            }

            # ==================== ✅ ASIGNACIÓN MÉDICO-PACIENTE (SIMPLIFICADO) ====================
            try:
                id_paciente_persona = request.usuario.id_persona_id

                # Usar la función helper para asignar médico
                asignacion, es_nueva = _asignar_medico_si_no_existe(
                    id_paciente_persona)

                # Obtener información del médico
                medico_info = _obtener_info_medico(asignacion.id_medico)

            except Exception as e:
                return Response(
                    {
                        'status': 'error',
                        'error': f'No se pudo asignar médico automáticamente: {str(e)}'
                    },
                    status=status.HTTP_200_OK
                )
            # ==================== FIN ASIGNACIÓN ====================

            # Predicción
            resultado_prediccion = dengue_service.predecir_dengue(
                datos_paciente)

            tiempo_respuesta = time.time() - inicio_tiempo
```

```python
        # Crear diagnóstico en base de datos
        diagnostico = Diagnostico(
            id_usuario=request.usuario,
            id_prueba=uuid.uuid4(),
            fecha_prueba=timezone.now(),
            edad=datos_paciente['edad'],
            sexo=datos_paciente['sexo'],
            ns1=datos_paciente['ns1'],
            igm=datos_paciente['igm'],
            igg=datos_paciente['igg'],
            plaquetas=datos_paciente['plaquetas'],
            hematocrito=datos_paciente['hematocrito'],
            resultado_enfermedad=resultado_prediccion['resultado_enfermedad'],
            severidad_signos=resultado_prediccion['severidad_signos'],
            tiempo_respuesta=tiempo_respuesta,
            recomendacion=resultado_prediccion['recomendaciones'],
            estado='0',
            url_imagen=url_imagen  # URL de Supabase Storage
        )
        diagnostico.save()

        serializer = DiagnosticoSerializer(diagnostico)

        # Preparar respuesta con información del médico
        response_data = {
            'status': 'success',
            'diagnostico': serializer.data,
            'asignacion_medico': {
                'asignacion_nueva': es_nueva,
                'medico': medico_info,
                'mensaje': 'Nuevo médico asignado' if es_nueva else 'Médico previamente asignado'
            },
            'storage_info': {
                'bucket': getattr(settings, "SUPABASE_BUCKET", "imagenes"),
                'uploaded': True
            }
        }

        return Response(response_data, status=status.HTTP_200_OK)

    finally:
        # Limpiar archivo temporal
        if default_storage.exists(temp_file_path):
            default_storage.delete(temp_file_path)

except ValueError as e:
    return Response(
        {
            'status': 'error',
            'error': f'Error en el formato de los datos: {str(e)}'
        },
        status=status.HTTP_200_OK
    )
```

Descripción: se muestra el código para el botón **Analizar Reporte.** Se crea una función para recibir una imagen de un reporte de análisis de dengue y a través del servicio de machine learning realiza el diagnóstico y además el servicio de inteligencia artificial de Gemini brinda recomendaciones personalizadas según el diagnóstico emitido.

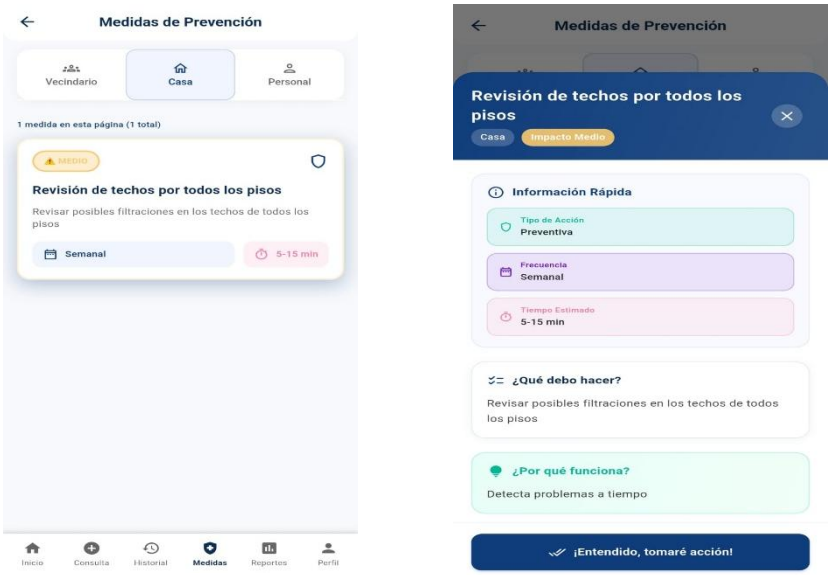
Pantalla historial de diagnósticos

```python
def listar_diagnosticos_usuario(request):
    """
    try:
        # ✅ 1) Verificar rol PACIENTE
        try:
            persona = Persona.objects.get(id_persona=request.usuario.id_persona_id)
        except Persona.DoesNotExist:
            return Response(
                {"status": "error", "error": "Persona no encontrada."},
                status=status.HTTP_200_OK
            )

        if not persona.id_rol or persona.id_rol.nombre_rol != "PACIENTE":
            return Response(
                {
                    "status": "error",
                    "error": "Solo los pacientes pueden acceder a este endpoint."
                },
                status=status.HTTP_200_OK
            )

        # ✅ 2) Traer diagnósticos del paciente
        diagnosticos_qs = Diagnostico.objects.filter(
            id_usuario=request.usuario
        ).order_by('-fecha_prueba')

        # ✅ 3) Traer validaciones (para estado 1 o 2) en un solo golpe
        from apps.diagnosticos.models import DiagnosticoValidacion

        diagnosticos_list = list(diagnosticos_qs)
        validaciones_qs = DiagnosticoValidacion.objects.filter(
            id_prueba__in=diagnosticos_list
        ).select_related("id_prueba")

        # Map: {uuid_prueba: validacion}
        validaciones_map = {}
        for v in validaciones_qs:
            try:
                validaciones_map[str(v.id_prueba_id)] = v
            except Exception:
                # fallback si tu modelo usa otra forma de id
                validaciones_map[str(v.id_prueba.id_prueba)] = v

        # ✅ 4) Serializar + enriquecer estructura para el frontend (Flutter)
        diagnosticos_final = []
        for d in diagnosticos_list:
            base = DiagnosticoSerializer(d).data

            estado = str(getattr(d, "estado", "") or "").strip()
            val = validaciones_map.get(str(d.id_prueba))

            # comentario_medico (para estado 1 o 2 puede existir)
            comentario = None
            if val and getattr(val, "comentario_medico", None) is not None:
                comentario = val.comentario_medico
```

```python
        diagnosticos_final = []
        for d in diagnosticos_list:
            base = DiagnosticoSerializer(d).data

            estado = str(getattr(d, "estado", "") or "").strip()
            val = validaciones_map.get(str(d.id_prueba))

            # comentario_medico (para estado 1 o 2 puede existir)
            comentario = None
            if val and getattr(val, "comentario_medico", None) is not None:
                comentario = val.comentario_medico

            # Para el frontend:
            # - resultado_medico / severidad_medica: SOLO deben ser "el correcto del médico" cuando es descartado.
            # - en confirmado, como el médico lo confirmó, final = lo de la app (pero igual lo exponemos como "medico").
            # - en pendiente, null.
            resultado_medico = None
            severidad_medica = None

            if estado == "2":
                # ✅ Descartado: final = corrección del médico
                resultado_medico = getattr(val, "resultado_enfermedad_correcto", None) if val else None
                severidad_medica = getattr(val, "severidad_signos_correcto", None) if val else None
            elif estado == "1":
                # ✅ Confirmado: final = lo predicho por la app
                resultado_medico = getattr(d, "resultado_enfermedad", None)
                severidad_medica = getattr(d, "severidad_signos", None)
            else:
                # ✅ Pendiente: aún no hay validación final
                resultado_medico = None
                severidad_medica = None

            base["comentario_medico"] = comentario
            base["resultado_medico"] = resultado_medico
            base["severidad_medica"] = severidad_medica

            diagnosticos_final.append(base)

        return Response(
            {
                "status": "success",
                "total_diagnosticos": len(diagnosticos_final),
                "diagnosticos": diagnosticos_final
            },
            status=status.HTTP_200_OK
        )

    except Exception as e:
        return Response(
            {
                "status": "error",
                "error": f"Error al obtener los diagnósticos: {str(e)}"
            },
            status=status.HTTP_200_OK
```

Descripción: se muestra el código para los botones de **Pendientes** y **Validados.** Se crea una función para listar todos los diagnósticos realizados por un paciente específico incluyendo sus detalles y la información si fueron validados o aún no.

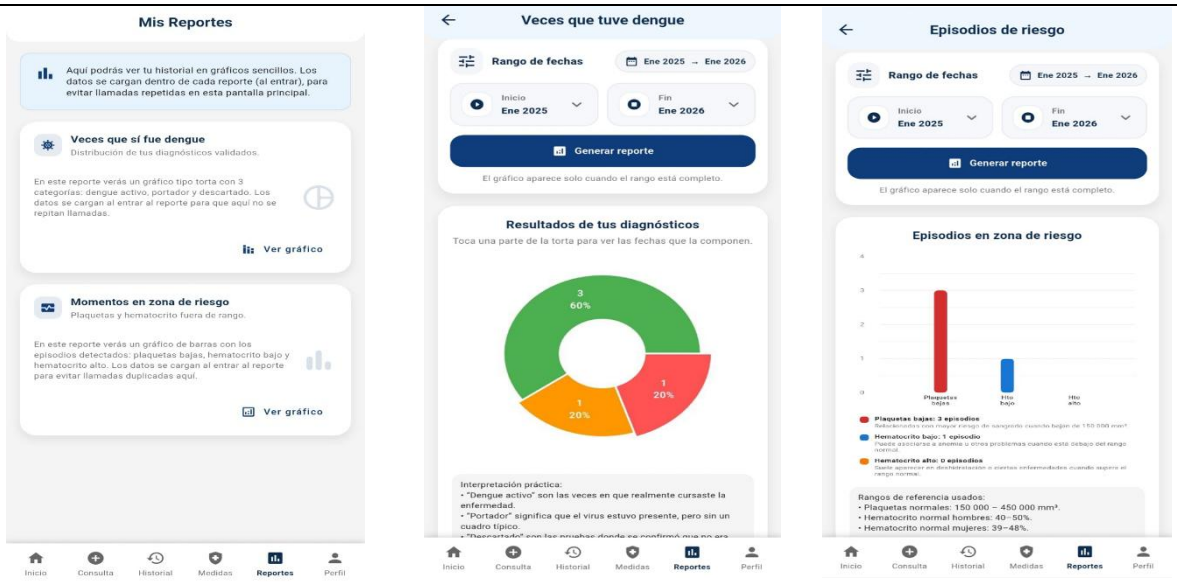
Pantalla de medidas preventivas




```python
def listar_medidas(request):
    categoria = request.GET.get('categoria')
    usuario = request.usuario

    # Si es administrador, mostrar todas las medidas
    # Si no es administrador, mostrar solo las activas
    es_admin = _es_admin(usuario)

    if es_admin:
        # ADMINISTRADOR: ve todas (activas e inactivas)
        qs = Medida.objects.all()
    else:
        # PACIENTE: solo ve las activas
        qs = Medida.objects.filter(activo=True)

    # Filtrar por categoría si se especifica
    if categoria:
        qs = qs.filter(categoria=categoria)

    # Ordenar por impacto y fecha de actualización
    qs = qs.order_by('impacto', '-updated_at')

    serializer = MedidaSerializer(qs, many=True)

    return Response(
        {
            "status": "success",
            "total": qs.count(),
            "admin": es_admin,  # Información adicional para el frontend
            "data": serializer.data
        },
        status=status.HTTP_200_OK
    )

# ==================================================
# VER DETALLE DE UNA MEDIDA (PACIENTE)
# ==================================================

@api_view(['GET'])
@permission_classes([TokenAuth])
def obtener_medida(request, id_medida):
    try:
        medida = Medida.objects.get(id_medida=id_medida, activo=True)
    except Medida.DoesNotExist:
        return Response(
            {"status": "error", "message": "Medida no encontrada."},
            status=status.HTTP_404_NOT_FOUND
        )

    serializer = MedidaSerializer(medida)

    return Response(
        {
            "status": "success",
            "data": serializer.data
```

Descripción: se muestra el código para los botones de las diferentes categorías de medidas**.** Se crean 2 funciones, una para listar todas las medidas y otra para mostrar el detalle completo de cada una de las medidas elegidas.

Pantalla de reportes de diagnósticos




```python
def reporte_paciente_resultados(request):
    try:
        # 1) Params obligatorios
        mes_inicio = request.GET.get("mes_inicio")
        anio_inicio = request.GET.get("anio_inicio")
        mes_fin = request.GET.get("mes_fin")
        anio_fin = request.GET.get("anio_fin")

        if not mes_inicio or not anio_inicio or not mes_fin or not anio_fin:
            return Response(
                {"status": "error", "error": "Parámetros obligatorios: mes_inicio, anio_inicio, mes_fin, anio_fin."},
                status=status.HTTP_200_OK
            )

        try:
            mes_inicio = int(str(mes_inicio).strip())
            anio_inicio = int(str(anio_inicio).strip())
            mes_fin = int(str(mes_fin).strip())
            anio_fin = int(str(anio_fin).strip())
        except Exception:
            return Response(
                {"status": "error", "error": "Los parámetros mes/año deben ser numéricos."},
                status=status.HTTP_200_OK
            )

        if not (1 <= mes_inicio <= 12) or not (1 <= mes_fin <= 12):
            return Response(
                {"status": "error", "error": "mes_inicio y mes_fin deben estar entre 1 y 12."},
                status=status.HTTP_200_OK
            )

        # 2) Rango fechas (>= inicio, < fin_exclusivo) en hora Perú
        from datetime import datetime
        from zoneinfo import ZoneInfo

        tz_pe = ZoneInfo("America/Lima")

        inicio_dt = datetime(anio_inicio, mes_inicio, 1, 0, 0, 0, tzinfo=tz_pe)
        if mes_fin == 12:
            fin_exclusivo = datetime(anio_fin + 1, 1, 1, 0, 0, 0, tzinfo=tz_pe)
        else:
            fin_exclusivo = datetime(anio_fin, mes_fin + 1, 1, 0, 0, 0, tzinfo=tz_pe)

        if inicio_dt >= fin_exclusivo:
            return Response(
                {"status": "error", "error": "El rango es inválido: inicio debe ser anterior a fin."},
                status=status.HTTP_200_OK
            )

        # 3) Normalizador LOCAL (ya no depende del helper global)
        def normalizar_resultado(valor) -> str:
            if not valor:
                return "Descartado"
            v = str(valor).strip().lower()
            if "activo" in v or "agudo" in v or "confirmado" in v:
```

```python
        def normalizar_resultado(valor) -> str:
            """Normalizar resultado"""
            v = str(valor).strip().lower()
            if "activo" in v or "agudo" in v or "confirmado" in v:
                return "Activo"
            if "portador" in v or "pasada" in v or "cronico" in v or "crónico" in v:
                return "Portador"
            if "descartado" in v or "negativo" in v or "no dengue" in v:
                return "Descartado"
            if "dengue" in v:
                return "Activo"
            return "Descartado"

        # 4) Query
        diagnosticos = Diagnostico.objects.filter(
            id_usuario=request.usuario,
            estado__in=["1", "2"],
            fecha_prueba__gte=inicio_dt,
            fecha_prueba__lt=fin_exclusivo,
        ).order_by("fecha_prueba")

        from apps.diagnosticos.models import DiagnosticoValidacion

        # Validaciones solo de descartados (estado=2) en 1 golpe
        validaciones_map = {}
        descartados_list = list(diagnosticos.filter(estado="2"))
        if descartados_list:
            validaciones = DiagnosticoValidacion.objects.filter(
                id_prueba__in=descartados_list
            ).select_related("id_prueba")

            for v in validaciones:
                try:
                    validaciones_map[str(v.id_prueba_id)] = v
                except Exception:
                    # fallback por si tu modelo expone el uuid de otra forma
                    try:
                        validaciones_map[str(v.id_prueba.id_prueba)] = v
                    except Exception:
                        pass

        # 5) Conteos + fechas agrupadas
        conteos = {"Activo": 0, "Portador": 0, "Descartado": 0}
        fechas_por_resultado = {"Activo": [], "Portador": [], "Descartado": []}

        for d in diagnosticos:
            if str(d.estado) == "2":
                val = validaciones_map.get(str(d.id_prueba))
                res_final = val.resultado_enfermedad_correcto if val else d.resultado_enfermedad
            else:
                res_final = d.resultado_enfermedad

            res_final = normalizar_resultado(res_final)
```

```python
            fecha_local = timezone.localtime(d.fecha_prueba, tz_pe)
            fecha_str = fecha_local.strftime("%d/%m/%Y %I:%M %p")  # AM/PM

            conteos[res_final] = conteos.get(res_final, 0) + 1
            fechas_por_resultado[res_final].append(fecha_str)

        return Response(
            {
                "status": "success",
                "total_diagnosticos_validados": diagnosticos.count(),
                "conteo_resultados": conteos,
                "fechas_por_resultado": fechas_por_resultado,
            },
            status=status.HTTP_200_OK
        )

    except Exception as e:
        return Response(
            {"status": "error", "error": f"Error al generar el reporte: {str(e)}"},
            status=status.HTTP_200_OK
        )


@api_view(['GET'])
@permission_classes([TokenAuth])
def reporte_paciente_riesgo(request):
    """
    Reporte INDIVIDUAL (paciente actual) entre un rango de meses/años.
    ✅ mes_inicio, anio_inicio, mes_fin, anio_fin son OBLIGATORIOS.
    ✅ Mantiene análisis de riesgo (plaquetas/hematocrito).
    ✅ Devuelve conteos + fechas agrupadas por tipo de riesgo encontrado.
    """
    try:
        # ----------------------------
        # 1) Validar query params (obligatorios)
        # ----------------------------
        mes_inicio = request.GET.get("mes_inicio", None)
        anio_inicio = request.GET.get("anio_inicio", None)
        mes_fin = request.GET.get("mes_fin", None)
        anio_fin = request.GET.get("anio_fin", None)

        if not mes_inicio or not anio_inicio or not mes_fin or not anio_fin:
            return Response(
                {
                    "status": "error",
                    "error": "Parámetros obligatorios: mes_inicio, anio_inicio, mes_fin, anio_fin."
                },
                status=status.HTTP_200_OK
            )
```

```python
        mes_fin = int(str(mes_fin).strip())
        anio_fin = int(str(anio_fin).strip())
    except Exception:
        return Response(
            {
                "status": "error",
                "error": "Los parámetros mes/año deben ser numéricos."
            },
            status=status.HTTP_200_OK
        )

    if mes_inicio < 1 or mes_inicio > 12 or mes_fin < 1 or mes_fin > 12:
        return Response(
            {
                "status": "error",
                "error": "mes_inicio y mes_fin deben estar entre 1 y 12."
            },
            status=status.HTTP_200_OK
        )

    # ---------------------------
    # 2) Construir rango de fechas (>= inicio, < fin_exclusivo)
    # ---------------------------
    tz_pe = ZoneInfo("America/Lima")

    inicio_dt = datetime(anio_inicio, mes_inicio, 1, 0, 0, 0, tzinfo=tz_pe)

    if mes_fin == 12:
        fin_exclusivo = datetime(anio_fin + 1, 1, 1, 0, 0, 0, tzinfo=tz_pe)
    else:
        fin_exclusivo = datetime(anio_fin, mes_fin + 1, 1, 0, 0, 0, tzinfo=tz_pe)

    if inicio_dt >= fin_exclusivo:
        return Response(
            {
                "status": "error",
                "error": "El rango es inválido: la fecha de inicio debe ser anterior o igual al rango de fin."
            },
            status=status.HTTP_200_OK
        )

    # ---------------------------
    # 3) Consultar diagnósticos del paciente en el rango
    # ---------------------------
    diagnosticos = Diagnostico.objects.filter(
        id_usuario=request.usuario,
        estado__in=["1", "2"],
        fecha_prueba__gte=inicio_dt,
        fecha_prueba__lt=fin_exclusivo,
    ).order_by("fecha_prueba")

    # ---------------------------
    # 4) Conteos + fechas agrupadas por tipo de riesgo
```

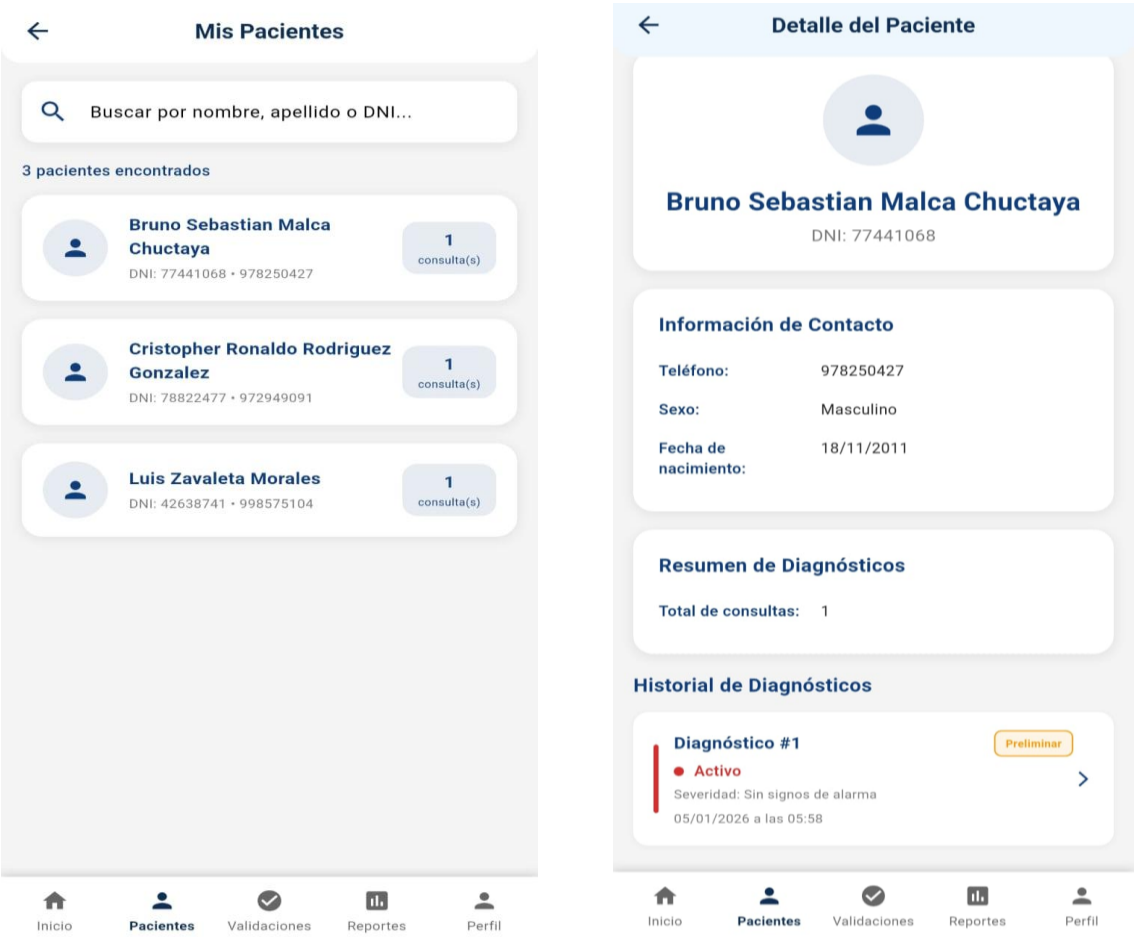
```python
    diagnosticos = Diagnostico.objects.filter(
        id_usuario=request.usuario,
        estado__in=["1", "2"],
        fecha_prueba__gte=inicio_dt,
        fecha_prueba__lt=fin_exclusivo,
    ).order_by("fecha_prueba")

    # ---------------------------
    # 4) Conteos + fechas agrupadas por tipo de riesgo
    # ---------------------------
    episodios_plaquetas_bajas = 0
    episodios_hto_bajo = 0
    episodios_hto_alto = 0

    fechas_riesgo = {
        "plaquetas_bajas": [],
        "hematocrito_bajo": [],
        "hematocrito_alto": [],
    }

    for d in diagnosticos:
        # Fecha Perú en formato solicitado
        fecha_local = timezone.localtime(d.fecha_prueba, tz_pe)
        fecha_str = fecha_local.strftime("%d/%m/%Y %I:%M %p")  # AM/PM

        # Plaquetas bajas
        if d.plaquetas is not None and d.plaquetas < 150000:
            episodios_plaquetas_bajas += 1
            fechas_riesgo["plaquetas_bajas"].append(fecha_str)

        # Hematocrito
        if d.hematocrito is not None:
            hto = float(d.hematocrito)

            if d.sexo == 'M':
                if hto < 40.0:
                    episodios_hto_bajo += 1
                    fechas_riesgo["hematocrito_bajo"].append(fecha_str)
                elif hto > 50.0:
                    episodios_hto_alto += 1
                    fechas_riesgo["hematocrito_alto"].append(fecha_str)

            elif d.sexo == 'F':
                if hto < 39.0:
                    episodios_hto_bajo += 1
                    fechas_riesgo["hematocrito_bajo"].append(fecha_str)
                elif hto > 48.0:
                    episodios_hto_alto += 1
                    fechas_riesgo["hematocrito_alto"].append(fecha_str)

    total_riesgo = episodios_plaquetas_bajas + episodios_hto_bajo + episodios_hto_alto
```

```
        return Response(
            {
                "status": "success",
                "total_diagnosticos_validados": diagnosticos.count(),
                "episodios_plaquetas_bajas": episodios_plaquetas_bajas,
                "episodios_hto_bajo": episodios_hto_bajo,
                "episodios_hto_alto": episodios_hto_alto,
                "total_episodios_riesgo": total_riesgo,
                "fechas_riesgo": fechas_riesgo,
            },
            status=status.HTTP_200_OK
        )

    except Exception as e:
        return Response(
            {
                "status": "error",
                "error": f"Error al generar el reporte: {str(e)}"
            },
            status=status.HTTP_200_OK
        )
```

Descripción: Se muestra el código para los dos botones para cada reporte definidos para los pacientes**.** Se crean dos funciones que cada uno retorna los datos necesarios para realizar los gráficos en la vista.

C. USUARIOS CON ROL "MÉDICO"

Pantalla listado de pacientes asignados



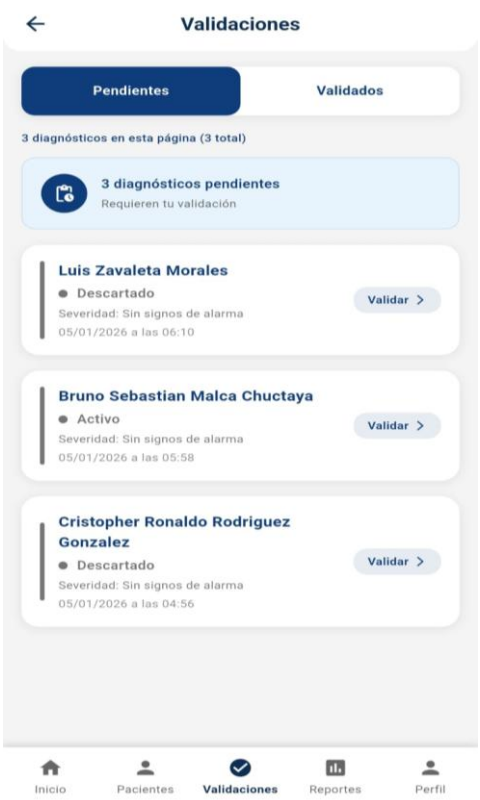
```python
def listar_pacientes_completos(request):
    """
        medico = Persona.objects.select_related("id_rol").filter(
            id_persona=request.usuario.id_persona_id
        ).first()

        if not medico or medico.id_rol.nombre_rol.upper() != "MÉDICO":
            return Response(
                {"status": "error", "message": "Solo los médicos pueden acceder a esta función."},
                status=status.HTTP_200_OK,
            )

        rol_paciente = Rol.objects.filter(nombre_rol__iexact="PACIENTE").first()
        if not rol_paciente:
            return Response(
                {"status": "error", "message": "No se encontró el rol PACIENTE."},
                status=status.HTTP_500_INTERNAL_SERVER_ERROR,
            )

        # Parámetros
        search_query = (request.GET.get("search") or "").strip()
        limit = int(request.GET.get("limit", 20))
        offset = int(request.GET.get("offset", 0))
        limit = max(1, min(limit, 200))

        sub_asignados = PacienteMedico.objects.filter(
            id_medico=medico.id_persona, activo=True
        ).values("id_paciente")

        correo_subquery = Usuario.objects.filter(
            id_persona=OuterRef("id_persona")
        ).values("correo")[:1]

        diagnosticos_prefetch = Prefetch(
            "usuario_set__diagnostico_set",
            queryset=Diagnostico.objects.only(
                "id_prueba", "id_usuario", "fecha_prueba", "edad", "sexo",
                "ns1", "igm", "igg", "plaquetas", "hematocrito",
                "resultado_enfermedad", "severidad_signos", "tiempo_respuesta",
                "recomendacion", "estado", "url_imagen",
            ).order_by("-fecha_prueba"),
            to_attr="diagnosticos_cache",
        )

        pacientes_qs = (
            Persona.objects.filter(
                id_persona__in=Subquery(sub_asignados),
                id_rol=rol_paciente.id_rol,
            )
            .annotate(correo=Subquery(correo_subquery))
            .select_related("id_rol")
            .prefetch_related("usuario_set", diagnosticos_prefetch)
        )
```

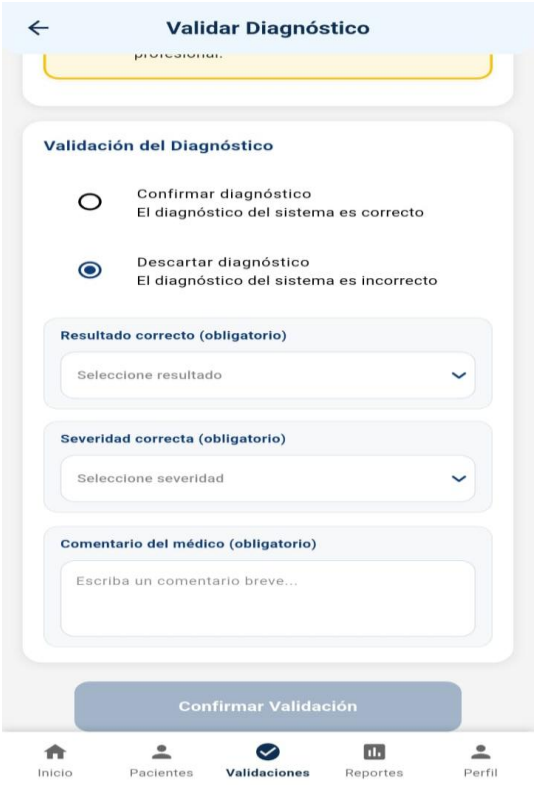
```python
    if search_query:
        pacientes_qs = pacientes_qs.filter(
            Q(nombres__icontains=search_query)
            | Q(apellidos__icontains=search_query)
            | Q(dni__icontains=search_query)
            | Q(celular__icontains=search_query)
        )

    total = pacientes_qs.count()
    pacientes_page = pacientes_qs.order_by("nombres", "apellidos")[offset : offset + limit]

    pacientes_data = []
    for paciente in pacientes_page:
        paciente_data = PacienteCompletoSerializer(paciente).data

        # Eliminar campos no deseados
        paciente_data.pop("rol", None)
        paciente_data.pop("ultimo_diagnostico", None)

        diagnosticos_usuario = []
        for usuario in paciente.usuario_set.all():
            diagnosticos_usuario.extend(getattr(usuario, "diagnosticos_cache", []))

        diagnostico_serializado = []
        for diag in diagnosticos_usuario:
            data = DiagnosticoConPacienteSerializer(diag).data

            # Validación médica
            validacion = getattr(diag, "validacion", None)
            if validacion:
                data["comentario_medico"] = validacion.comentario_medico
                data["resultado_medico"] = validacion.resultado_enfermedad_correcto
                data["severidad_medica"] = validacion.severidad_signos_correcto
            else:
                data["comentario_medico"] = None
                data["resultado_medico"] = None
                data["severidad_medica"] = None

            # Reglas de presentación
            if diag.estado == "0":
                data["mostrar"] = {
                    "comentario": False,
                    "recomendacion": False,
                    "mensaje": "Diagnóstico pendiente de validación médica."
                }
            elif diag.estado == "1":
                data["mostrar"] = {
                    "comentario": True,
                    "recomendacion": True,
                    "mensaje": "Diagnóstico confirmado por el médico."
                }
            elif diag.estado == "2":
                data["recomendacion"] = None
                data["mostrar"] = {
```

```python
            elif diag.estado == "2":
                data["recomendacion"] = None
                data["mostrar"] = {
                    "comentario": True,
                    "recomendacion": False,
                    "mensaje": "Diagnóstico descartado por el médico."
                }

            diagnostico_serializado.append(data)

        paciente_data["diagnosticos"] = diagnostico_serializado
        pacientes_data.append(paciente_data)

    return Response(
        {
            "status": "success",
            "total_pacientes": total,
            "rol_filtrado": rol_paciente.nombre_rol,
            "medico_id": str(medico.id_persona),
            "pacientes": pacientes_data,
        },
        status=status.HTTP_200_OK,
    )

except Exception as e:
    return Response(
        {"status": "error", "message": f"Error al obtener los pacientes: {str(e)}"},
        status=status.HTTP_500_INTERNAL_SERVER_ERROR,
    )
```

Descripción: se muestra el código para poder obtener la lista de pacientes asignados a un médico específico. Se crea una función para recuperar los datos completos de los pacientes que tiene a cargo el médico incluyendo los datos de sus diagnósticos realizados.

Pantalla de Validación de diagnósticos





```python
def validar_diagnostico(request, id_prueba):
    # ✅ 1) Verificar que el usuario sea MÉDICO
    try:
        persona = Persona.objects.get(id_persona=request.usuario.id_persona_id)
        if persona.id_rol.nombre_rol != 'MÉDICO':
            return Response(
                {"status": "error",
                    "message": "Solo los médicos pueden validar diagnósticos."},
                status=status.HTTP_200_OK
            )
    except Persona.DoesNotExist:
        return Response(
            {"status": "error", "message": "Persona no encontrada."},
            status=status.HTTP_200_OK
        )

    # ✅ 2) Buscar diagnóstico
    try:
        diagnostico = Diagnostico.objects.get(id_prueba=id_prueba)
    except Diagnostico.DoesNotExist:
        return Response(
            {"status": "error", "message": "Diagnóstico no encontrado."},
            status=status.HTTP_200_OK
        )

    # ✅ 3) Solo se puede validar si está pendiente
    if diagnostico.estado != '0':
        return Response(
            {"status": "error", "message": "El diagnóstico ya ha sido validado."},
            status=status.HTTP_200_OK
        )

    # ✅ 4) Verificar que el diagnóstico pertenezca a un paciente asignado a este médico
    try:
        id_paciente_persona = diagnostico.id_usuario.id_persona_id if diagnostico.id_usuario else None
        if not id_paciente_persona:
            return Response(
                {"status": "error",
                    "message": "No se pudo identificar al paciente del diagnóstico."},
                status=status.HTTP_200_OK
            )

        asignacion = _obtener_medico_asignado_activo(id_paciente_persona)

        if not asignacion or asignacion.id_medico_id != persona.id_persona:
            return Response(
                {"status": "error", "message": "No puedes validar diagnósticos de pacientes que no tienes asignados."},
                status=status.HTTP_200_OK
            )
    except Exception as e:
        return Response(
            {"status": "error",
                "message": f"Error verificando asignación: {str(e)}"},
            status=status.HTTP_200_OK
        )
```

carlosgs05 (Hace 2 semanas)

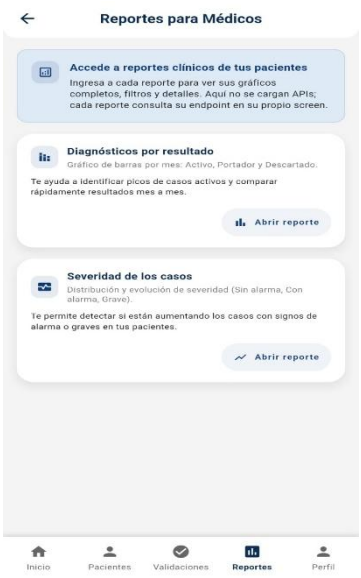
```python
# ✅ 5) Leer payload
data = request.data
nuevo_estado = str(data.get("estado", "")).strip()

# comentario (lo guardamos tanto para confirma como para descarta)
comentario_medico = data.get("comentario_medico", None)

# correcciones (solo para descarta)
resultado_correcto = data.get("resultado_enfermedad_correcto", None)
severidad_correcta = data.get("severidad_signos_correcto", None)

# ✅ 6) Validar estado
if nuevo_estado not in ["1", "2"]:
    return Response(
        {"status": "error", "message": "Estado inválido. Use '1' para confirmado o '2' para descartado."},
        status=status.HTTP_200_OK
    )

# ✅ 7) Reglas de campos según estado
opciones_resultado = ["Activo", "Portador", "Descartado"]
opciones_severidad = ["Sin signos de alarma",
                      "Con signos de alarma", "Grave"]

if nuevo_estado == "1":
    # Confirma: NO debe mandar correcciones
    if resultado_correcto is not None or severidad_correcta is not None:
        return Response(
            {
                "status": "error",
                "message": "Si confirmas (estado=1), no debes enviar resultado_enfermedad_correcto ni severidad_signos_correcto."
            },
            status=status.HTTP_200_OK
        )

if nuevo_estado == "2":
    # Descarta: DEBE mandar correcciones
    if not resultado_correcto or not severidad_correcta:
        return Response(
            {
                "status": "error",
                "message": "Si descartas (estado=2), debes enviar resultado_enfermedad_correcto y severidad_signos_correcto."
            },
            status=status.HTTP_200_OK
        )

    # Validar que cumplan las opciones permitidas (por las restricciones de tu tabla de entrenamiento)
    if str(resultado_correcto).strip() not in opciones_resultado:
        return Response(
            {
                "status": "error",
                "message": f"resultado_enfermedad_correcto inválido. Use: {', '.join(opciones_resultado)}"
            },
            status=status.HTTP_200_OK
        )
```

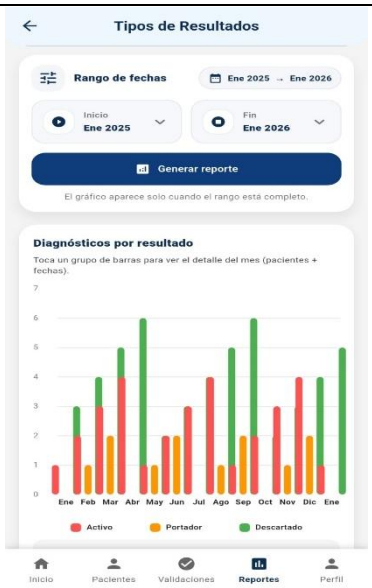
```python
        # 8.1) Actualizar estado del diagnóstico
        diagnostico.estado = nuevo_estado
        diagnostico.save()

        # 8.2) Guardar/actualizar validación médica en diagnostico_validacion
        #      (1 registro por diagnóstico)
        # OJO: necesitas tener importado DiagnosticoValidacion en este archivo
        from apps.diagnosticos.models import DiagnosticoValidacion

        if nuevo_estado == "1":
            # Confirmado: solo comentario, correcciones en NULL
            DiagnosticoValidacion.objects.update_or_create(
                id_prueba=diagnostico,
                defaults={
                    "id_medico": persona,
                    "comentario_medico": comentario_medico,
                    "resultado_enfermedad_correcto": None,
                    "severidad_signos_correcto": None,
                }
            )

            # Datos que irán a entrenamiento = lo predicho por la app
            resultado_entrenamiento = diagnostico.resultado_enfermedad
            severidad_entrenamiento = diagnostico.severidad_signos

        else:
            # Descartado: guarda correcciones + comentario
            DiagnosticoValidacion.objects.update_or_create(
                id_prueba=diagnostico,
                defaults={
                    "id_medico": persona,
                    "comentario_medico": comentario_medico,
                    "resultado_enfermedad_correcto": str(resultado_correcto).strip(),
                    "severidad_signos_correcto": str(severidad_correcta).strip(),
                }
            )

            # Datos que irán a entrenamiento = lo corregido por el médico
            resultado_entrenamiento = str(resultado_correcto).strip()
            severidad_entrenamiento = str(severidad_correcta).strip()

        # 8.3) Guardar SIEMPRE en tabla de entrenamiento (confirmado o descartado)
        RegistrosDengueML.objects.create(
            edad=diagnostico.edad,
            sexo=diagnostico.sexo,
            ns1=diagnostico.ns1,
            igm=diagnostico.igm,
            igg=diagnostico.igg,
            plaquetas=diagnostico.plaquetas,
            hematocrito=diagnostico.hematocrito,
            resultado_enfermedad=resultado_entrenamiento,
            severidad_signos=severidad_entrenamiento
        )
```

```python
        # 8.4) Reentrenamiento automático (MISMA lógica que ya tenías)
        try:
            total_actual = dengue_service._get_total_registros_entrenamiento()
            meta = dengue_service._leer_meta_entrenamiento()
            total_anterior = int(meta.get("total_registros", 0))
            umbral = 50

            if total_actual - total_anterior >= umbral:
                print(
                    f"⚙️ Se detectaron {total_actual - total_anterior} nuevos registros.")
                print("🧠 Reentrenando modelo automáticamente...")
                dengue_service.entrenar_modelo()
                dengue_service._guardar_meta_entrenamiento(total_actual)
                print("✅ Reentrenamiento completado correctamente.")
        except Exception as e:
            print(f"⚠️ No se pudo reentrenar el modelo: {e}")

        # Respuesta
        return Response(
            {
                "status": "success",
                "message": "Diagnóstico validado exitosamente.",
                "diagnostico": DiagnosticoConPacienteSerializer(diagnostico).data,
                "validacion_medica": {
                    "comentario_medico": comentario_medico,
                    "resultado_enfermedad_final": resultado_entrenamiento,
                    "severidad_signos_final": severidad_entrenamiento,
                }
            },
            status=status.HTTP_200_OK
        )

    except Exception as e:
        return Response(
            {"status": "error",
                "message": f"Error al validar el diagnóstico: {str(e)}"},
            status=status.HTTP_200_OK
        )
```

Descripción: se muestra el código para el botón **Confirmar validación.** Se crea una función para confirmar o corregir el diagnóstico dado por la app, el médico coloca un comentario y además si tiene que corregir coloca los resultados correctos. Luego el sistema almacena los datos del diagnóstico correcto en una tabla de la base de datos para que el sistema puede entrenar con cada diagnóstico validado nuevo.

Pantalla de Reportes de Pacientes





```python
def reporte_medico_resultados(request):
    """
    try:
        persona = Persona.objects.get(id_persona=request.usuario.id_persona_id)
        if persona.id_rol.nombre_rol != 'MÉDICO':
            return Response(
                {
                    "status": "error",
                    "message": "Solo los médicos pueden acceder a los reportes globales.",
                }
            )
    except Persona.DoesNotExist:
        return Response({"status": "error", "message": "Persona no encontrada."})

    # ==========================
    # Filtros de fechas (mes/año inicio y fin)
    # ==========================
    mes_inicio = request.GET.get("mes_inicio")
    anio_inicio = request.GET.get("anio_inicio")
    mes_fin = request.GET.get("mes_fin")
    anio_fin = request.GET.get("anio_fin")

    tz = timezone.get_current_timezone()

    if mes_inicio and anio_inicio and mes_fin and anio_fin:
        try:
            mi = int(mes_inicio)
            ai = int(anio_inicio)
            mf = int(mes_fin)
            af = int(anio_fin)

            if not (1 <= mi <= 12 and 1 <= mf <= 12):
                return Response(
                    {
                        "status": "error",
                        "message": "mes_inicio y mes_fin deben estar entre 1 y 12.",
                    }
                )

            inicio_naive = datetime(ai, mi, 1, 0, 0, 0)
            inicio = timezone.make_aware(inicio_naive, tz)

            last_day = monthrange(af, mf)[1]
            fin_naive = datetime(af, mf, last_day, 23, 59, 59, 999999)
            fin = timezone.make_aware(fin_naive, tz)

            if fin < inicio:
                return Response(
                    {
                        "status": "error",
                        "message": "El rango es inválido: el fin no puede ser menor que el inicio.",
                    }
                )
        except Exception:
            return Response(
```

```python
    else:
        # Compatibilidad: si no mandan rango, usar últimos N meses como antes
        meses_int = int(request.GET.get("meses", 6))
        ahora = timezone.now()
        fin = ahora
        inicio = ahora - timedelta(days=30 * meses_int)

    # ==========================
    # Query
    # ==========================
    qs = (
        Diagnostico.objects.filter(
            fecha_prueba__gte=inicio,
            fecha_prueba__lte=fin,
            estado__in=["1", "2"],
            id_usuario__id_persona__asignaciones_como_paciente__activo=True,
            id_usuario__id_persona__asignaciones_como_paciente__id_medico=persona,
        )
        .select_related("id_usuario", "id_usuario__id_persona")
        .order_by("fecha_prueba")
    )

    # ==========================
    # Helpers: nombre completo + fecha formateada
    # ==========================
    def _nombre_completo_persona(p):
        if p is None:
            return "Paciente"
        nombres = (getattr(p, "nombres", None) or getattr(p, "nombre", None) or "").strip()
        ap_pat = (getattr(p, "apellido_paterno", None) or "").strip()
        ap_mat = (getattr(p, "apellido_materno", None) or "").strip()
        apes = (getattr(p, "apellidos", None) or "").strip()

        if (ap_pat or ap_mat) and nombres:
            full = f"{nombres} {ap_pat} {ap_mat}".strip()
            return " ".join(full.split())

        if nombres and apes:
            full = f"{nombres} {apes}".strip()
            return " ".join(full.split())

        if nombres:
            return " ".join(nombres.split())

        if apes:
            return " ".join(apes.split())

        if ap_pat or ap_mat:
            full = f"{ap_pat} {ap_mat}".strip()
            return " ".join(full.split())

        return "Paciente"
```

```python
    def _fecha_completa_am_pm(dt):
        return local_dt.strftime("%d/%m/%Y %I:%M %p").lower()

    # ==========================
    # Agregación por mes + detalles por mes/resultado
    # ==========================
    resumen = {}
    detalles_por_mes_resultado = {}

    for d in qs:
        if d.estado == "2":
            val = DiagnosticoValidacion.objects.filter(id_prueba=d).first()
            resultado = val.resultado_enfermedad_correcto if val else d.resultado_enfermedad
        else:
            resultado = d.resultado_enfermedad

        if resultado not in ["Activo", "Portador", "Descartado"]:
            continue

        fecha_local = timezone.localtime(d.fecha_prueba, tz)
        mes_key = fecha_local.strftime("%Y-%m")

        if mes_key not in resumen:
            resumen[mes_key] = {"Activo": 0, "Portador": 0, "Descartado": 0}
        resumen[mes_key][resultado] += 1

        if mes_key not in detalles_por_mes_resultado:
            detalles_por_mes_resultado[mes_key] = {
                "Activo": [],
                "Portador": [],
                "Descartado": [],
            }

        paciente_persona = getattr(d.id_usuario, "id_persona", None)
        detalles_por_mes_resultado[mes_key][resultado].append(
            {
                "paciente": _nombre_completo_persona(paciente_persona),
                "fecha": _fecha_completa_am_pm(d.fecha_prueba),
            }
        )

    # ==========================
    # Formato final para gráfico
    # ==========================
    meses_labels = []
    activos, portadores, descartados = [], [], []
    nombres_meses = ["Ene", "Feb", "Mar", "Abr", "May", "Jun", "Jul", "Ago", "Sep", "Oct", "Nov", "Dic"]

    for mes in sorted(resumen.keys()):
        anio, mnum = mes.split("-")
        meses_labels.append(f"{nombres_meses[int(mnum) - 1]}")
        activos.append(resumen[mes]["Activo"])
        portadores.append(resumen[mes]["Portador"])
        descartados.append(resumen[mes]["Descartado"])
```

```python
    for mes in sorted(resumen.keys()):
        anio, mnum = mes.split("-")
        meses_labels.append(f"{nombres_meses[int(mnum) - 1]}")
        activos.append(resumen[mes]["Activo"])
        portadores.append(resumen[mes]["Portador"])
        descartados.append(resumen[mes]["Descartado"])

    return Response(
        {
            "status": "success",
            "meses": meses_labels,
            "datos_resultados": {
                "Activo": activos,
                "Portador": portadores,
                "Descartado": descartados,
            },
            "total_diagnosticos": sum(activos) + sum(portadores) + sum(descartados),
            "detalles_por_mes_resultado": detalles_por_mes_resultado,
        }
    )


@api_view(["GET"])
@permission_classes([TokenAuth])
def reporte_medico_severidad(request):
    """
    Reporte global para MÉDICO (tendencia mensual por severidad).

    ✅ Usa severidad corregida de DiagnosticoValidacion si el diagnóstico fue descartado (estado='2').
    ✅ Filtra por rango mes/año: mes_inicio, anio_inicio, mes_fin, anio_fin (recomendado).
       - Si NO se envía rango, usa compatibilidad: ?meses=N (default 12) hacia atrás desde hoy.
    ✅ Solo incluye diagnósticos VALIDADOS (estado 1 o 2) de pacientes ASIGNADOS a este médico.
    ✅ Devuelve:
       - meses (lista ordenada con year/month/key/label) -> SOLO meses con data
       - conteos por severidad (3 series para el gráfico) -> SOLO meses con data
       - detalles_por_mes_severidad: por cada mes y severidad -> lista de {paciente, fecha, id_prueba}
    """

    try:
        persona = Persona.objects.get(id_persona=request.usuario.id_persona_id)
        if not persona.id_rol or persona.id_rol.nombre_rol != "MÉDICO":
            return Response(
                {"status": "error", "message": "Solo los médicos pueden acceder a los reportes globales."},
                status=status.HTTP_200_OK
            )
    except Persona.DoesNotExist:
        return Response(
            {"status": "error", "message": "Persona no encontrada."},
            status=status.HTTP_200_OK
        )

    # ==========================
    # Timezone Perú
    # ==========================
```

```python
def reporte_medico_severidad(request):
    # ==========================
    # Filtros de fechas (mes/año inicio y fin)
    # ==========================
    mes_inicio = request.GET.get("mes_inicio")
    anio_inicio = request.GET.get("anio_inicio")
    mes_fin = request.GET.get("mes_fin")
    anio_fin = request.GET.get("anio_fin")

    if mes_inicio and anio_inicio and mes_fin and anio_fin:
        try:
            mi = int(str(mes_inicio).strip())
            ai = int(str(anio_inicio).strip())
            mf = int(str(mes_fin).strip())
            af = int(str(anio_fin).strip())

            if not (1 <= mi <= 12 and 1 <= mf <= 12):
                return Response(
                    {"status": "error", "message": "mes_inicio y mes_fin deben estar entre 1 y 12."},
                    status=status.HTTP_200_OK
                )

            inicio = datetime(ai, mi, 1, 0, 0, 0, tzinfo=tz_pe)

            last_day = monthrange(af, mf)[1]
            fin = datetime(af, mf, last_day, 23, 59, 59, 999999, tzinfo=tz_pe)

            if fin < inicio:
                return Response(
                    {"status": "error", "message": "El rango es inválido: el fin no puede ser menor que el inicio."},
                    status=status.HTTP_200_OK
                )
        except Exception:
            return Response(
                {
                    "status": "error",
                    "message": "Parámetros de fecha inválidos. Usa mes_inicio, anio_inicio, mes_fin, anio_fin como enteros."
                },
                status=status.HTTP_200_OK
            )
        modo_filtro = "rango"
    else:
        # Compatibilidad: últimos N meses hacia atrás (default 12)
        try:
            meses_int = int(request.GET.get("meses", 12))
        except Exception:
            meses_int = 12

        ahora = timezone.now().astimezone(tz_pe)
        y = ahora.year
        m = ahora.month - (meses_int - 1)
        while m <= 0:
            m += 12
            y -= 1
        inicio = datetime(y, m, 1, 0, 0, 0, tzinfo=tz_pe)
```

```python
        tzinfo=tz_pe
    )
    modo_filtro = "ultimos_meses"

# ==========================
# Helpers
# ==========================
nombres_meses = ["Ene", "Feb", "Mar", "Abr", "May", "Jun", "Jul", "Ago", "Sep", "Oct", "Nov", "Dic"]

def _nombre_completo_persona(p):
    if p is None:
        return "Paciente"
    nombres = (getattr(p, "nombres", None) or getattr(p, "nombre", None) or "").strip()
    ap_pat = (getattr(p, "apellido_paterno", None) or "").strip()
    ap_mat = (getattr(p, "apellido_materno", None) or "").strip()
    apes = (getattr(p, "apellidos", None) or "").strip()

    if (ap_pat or ap_mat) and nombres:
        full = f"{nombres} {ap_pat} {ap_mat}".strip()
        return " ".join(full.split())

    if nombres and apes:
        full = f"{nombres} {apes}".strip()
        return " ".join(full.split())

    if nombres:
        return " ".join(nombres.split())

    if apes:
        return " ".join(apes.split())

    if ap_pat or ap_mat:
        full = f"{ap_pat} {ap_mat}".strip()
        return " ".join(full.split())

    return "Paciente"

def _fecha_am_pm(dt):
    try:
        local_dt = dt.astimezone(tz_pe)
    except Exception:
        local_dt = timezone.localtime(dt, tz_pe)
    return local_dt.strftime("%d/%m/%Y %I:%M %p").lower()

def _normalizar_severidad(valor: str) -> str:
    """
    Normaliza a una de estas 3:
    - 'Sin signos de alarma'
    - 'Con signos de alarma'
    - 'Grave'
    """
    if not valor:
        return ""
    v = str(valor).strip().lower()
```

```python
def _normalizar_severidad(valor: str) -> str:
    if "grave" in v:
        return "Grave"
    if "con" in v and "alar" in v:
        return "Con signos de alarma"
    if "sin" in v and "alar" in v:
        return "Sin signos de alarma"
    if v == "con signos de alarma":
        return "Con signos de alarma"
    if v == "sin signos de alarma":
        return "Sin signos de alarma"
    return ""

def _iter_months(start_dt, end_dt):
    """
    Genera meses inclusivos entre start_dt y end_dt en timezone Perú.
    """
    y, m = start_dt.year, start_dt.month
    end_y, end_m = end_dt.year, end_dt.month

    res = []
    while (y < end_y) or (y == end_y and m <= end_m):
        key = f"{y:04d}-{m:02d}"
        label = f"{nombres_meses[m - 1]} {y}"
        res.append({"year": y, "month": m, "key": key, "label": label})
        m += 1
        if m == 13:
            m = 1
            y += 1
    return res

meses = _iter_months(inicio, fin)

# ==========================
# IDs posibles del médico en asignación
# ==========================
medico_ids = []
try:
    medico_ids.append(persona.pk)
except Exception:
    pass
try:
    medico_ids.append(request.usuario.pk)
except Exception:
    pass
try:
    medico_ids.append(request.usuario.id_persona_id)
except Exception:
    pass
medico_ids = [x for x in medico_ids if x is not None]

# ==========================
# Query
# ==========================
```

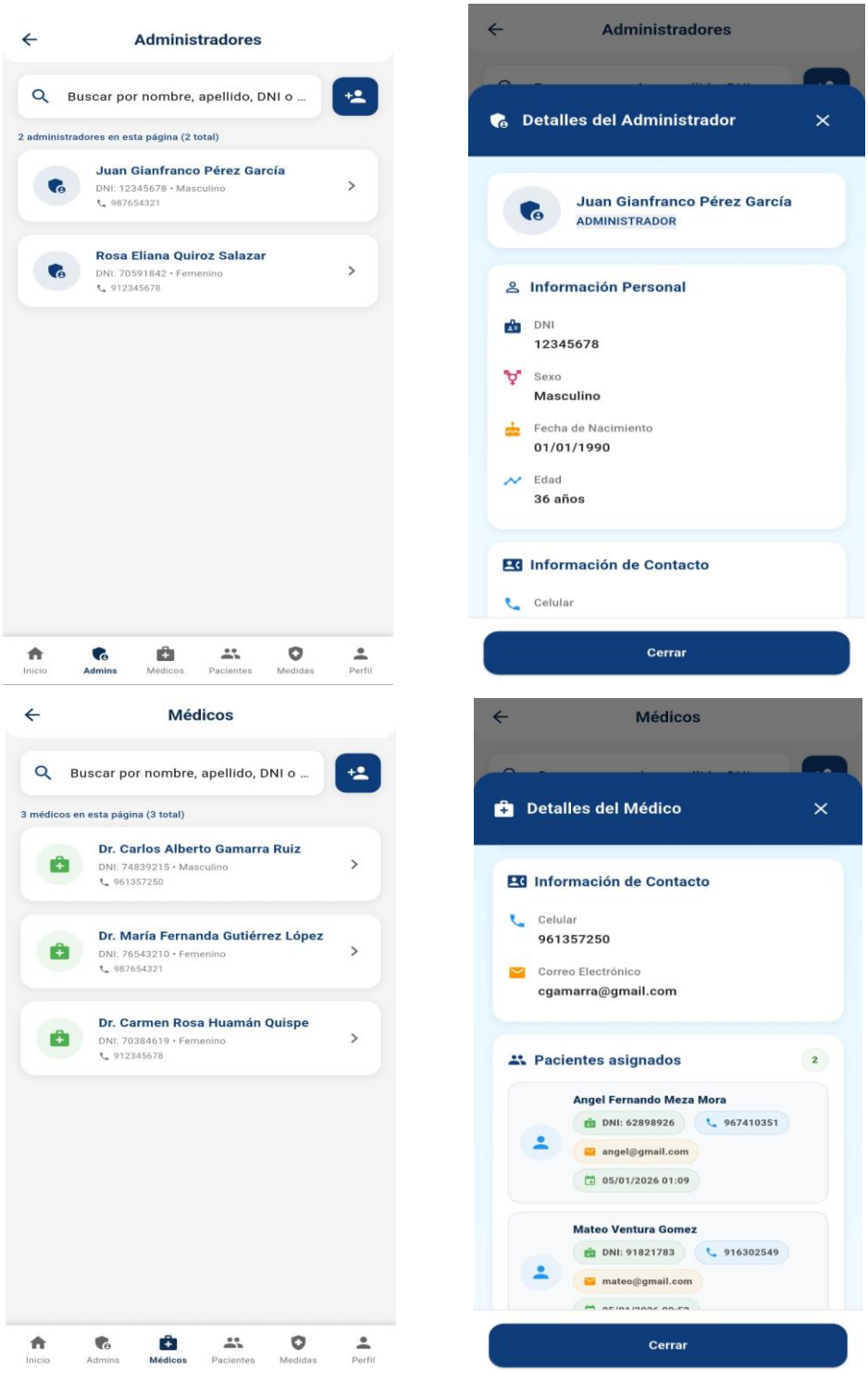
```python
qs = (
    Diagnostico.objects.filter(
        fecha_prueba__isnull=False,
        fecha_prueba__gte=inicio,
        fecha_prueba__lte=fin,
        estado__in=[1, 2, "1", "2"],
        id_usuario__isnull=False,
        id_usuario__id_persona__isnull=False,
        id_usuario__id_persona__asignaciones_como_paciente__activo=True,
        id_usuario__id_persona__asignaciones_como_paciente__id_medico_id__in=medico_ids,
    )
    .select_related("id_usuario", "id_usuario__id_persona", "validacion")
    .order_by("fecha_prueba")
    .distinct()
)

# ==========================
# Base por mes
# ==========================
conteo_por_mes = {
    m["key"]: {
        "Sin signos de alarma": 0,
        "Con signos de alarma": 0,
        "Grave": 0,
    }
    for m in meses
}

detalles_por_mes_severidad = {
    m["key"]: {
        "Sin signos de alarma": [],
        "Con signos de alarma": [],
        "Grave": [],
    }
    for m in meses
}

# ==========================
# Agregación
# ==========================
for d in qs:
    try:
        dt_local = d.fecha_prueba.astimezone(tz_pe)
    except Exception:
        dt_local = timezone.localtime(d.fecha_prueba, tz_pe)
    mes_key = dt_local.strftime("%Y-%m")

    sev_final = getattr(d, "severidad_signos", None)
    if str(getattr(d, "estado", "")) == "2":
        val = getattr(d, "validacion", None)
        if val and getattr(val, "severidad_signos_correcto", None):
            sev_final = val.severidad_signos_correcto

    sev_final = _normalizar_severidad(sev_final)
```

```python
    )

# ==========================
# FILTRO: quedarnos SOLO con meses que tienen data (cualquier severidad)
# ==========================
meses_con_data = []
for m in meses:
    key = m["key"]
    total_mes = (
        conteo_por_mes[key]["Sin signos de alarma"]
        + conteo_por_mes[key]["Con signos de alarma"]
        + conteo_por_mes[key]["Grave"]
    )
    if total_mes > 0:
        meses_con_data.append(m)

# Re-armar estructuras SOLO con meses_con_data
sin_signos = []
con_alarma = []
grave = []
detalles_filtrados = {}

for m in meses_con_data:
    key = m["key"]
    sin_signos.append(conteo_por_mes[key]["Sin signos de alarma"])
    con_alarma.append(conteo_por_mes[key]["Con signos de alarma"])
    grave.append(conteo_por_mes[key]["Grave"])
    detalles_filtrados[key] = detalles_por_mes_severidad[key]

total = sum(sin_signos) + sum(con_alarma) + sum(grave)

return Response(
    {
        "status": "success",
        "filtro": {
            "modo": modo_filtro,
            "mes_inicio": inicio.month,
            "anio_inicio": inicio.year,
            "mes_fin": fin.month,
            "anio_fin": fin.year,
            "fecha_inicio_iso": inicio.isoformat(),
            "fecha_fin_iso": fin.isoformat(),
        },
        "meses": meses_con_data,  # ✅ SOLO meses con data
        "datos_severidad": {
            "Sin signos de alarma": sin_signos,  # ✅ SOLO meses con data
            "Con signos de alarma": con_alarma,  # ✅ SOLO meses con data
            "Grave": grave,                      # ✅ SOLO meses con data
        },
        "total_diagnosticos": total,
        "detalles_por_mes_severidad": detalles_filtrados,  # ✅ SOLO meses con data
    },
    status=status.HTTP_200_OK
)
```

Descripción: Se muestra el código para los dos botones para cada reporte definidos para los médicos. Se crean dos funciones que cada uno retorna los datos necesarios para realizar los gráficos en la vista.

D. USUARIOS CON ROL "ADMINISTRADOR"

Pantalla de Administradores y Médicos

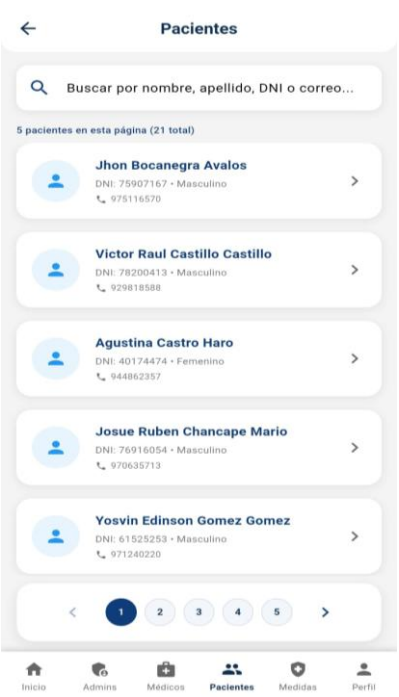


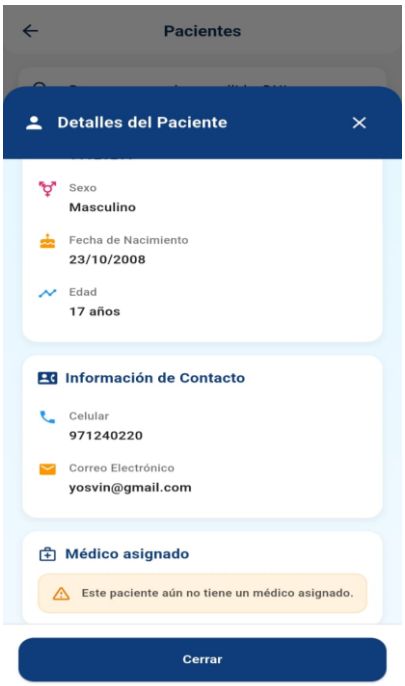
```python
def register_staff(request):
    try:
        persona_admin = Persona.objects.select_related("id_rol").get(id_persona=request.usuario.id_persona_id)
        if _strip_accents_upper(persona_admin.id_rol.nombre_rol) != "ADMINISTRADOR":
            return Response(
                {"status": "error", "message": "Solo el ADMINISTRADOR puede registrar personal."},
                status=status.HTTP_403_FORBIDDEN
            )
    except Persona.DoesNotExist:
        return Response(
            {"status": "error", "message": "Persona no encontrada para el usuario autenticado."},
            status=status.HTTP_400_BAD_REQUEST
        )

    data = request.data or {}

    try:
        norm = _validar_campos_register(data)
    except ValueError as e:
        return Response({"status": "error", "message": str(e)}, status=status.HTTP_400_BAD_REQUEST)

    nombre_rol = str(data.get("nombre_rol") or "").strip().upper()
    if nombre_rol == "MEDICO":
        nombre_rol = "MÉDICO"

    if nombre_rol not in ["MÉDICO", "ADMINISTRADOR"]:
        return Response(
            {"status": "error", "message": "Rol inválido. Solo se permite 'MÉDICO' o 'ADMINISTRADOR'."},
            status=status.HTTP_400_BAD_REQUEST
        )

    try:
        rol_destino = Rol.objects.get(nombre_rol__iexact=nombre_rol)
    except Rol.DoesNotExist:
        return Response(
            {"status": "error", "message": f"No existe el rol {nombre_rol} en la base de datos."},
            status=status.HTTP_500_INTERNAL_SERVER_ERROR
        )

    if Usuario.objects.filter(correo__iexact=norm["correo"]).exists():
        return Response({"status": "error", "message": "El correo ya está registrado."}, status=status.HTTP_409_CONFLICT)

    if Persona.objects.filter(dni=norm["dni"]).exists():
        return Response({"status": "error", "message": "El DNI ya está registrado."}, status=status.HTTP_409_CONFLICT)

    try:
        with transaction.atomic():
            persona = Persona.objects.create(
                id_persona=uuid.uuid4(),
                nombres=norm["nombres"],
                apellidos=norm["apellidos"],
                dni=norm["dni"],
                sexo=norm["sexo"],
                fecha_nacimiento=norm["fecha_nacimiento"],
                celular=norm["celular"],
```

```python
            usuario = Usuario.objects.create(
                id_usuario=uuid.uuid4(),
                id_persona=persona,
                correo=norm["correo"],
                contrasena=make_password(norm["contrasena"]),
            )

            return Response(
                {
                    "status": "success",
                    "message": f"Cuenta creada exitosamente ({rol_destino.nombre_rol}).",
                    "id_persona": str(persona.id_persona),
                    "id_usuario": str(usuario.id_usuario),
                },
                status=status.HTTP_201_CREATED,
            )
    except Exception as e:
        return Response(
            {"status": "error", "message": f"No se pudo registrar: {str(e)}"},
            status=status.HTTP_500_INTERNAL_SERVER_ERROR
        )
```

Descripción: se muestra el código para el botón **Registrar** tanto Administradores como Médicos**.** Se crea una función para registrar el staff mandando como parámetro el rol a registrar.

Pantalla de Listado de pacientes

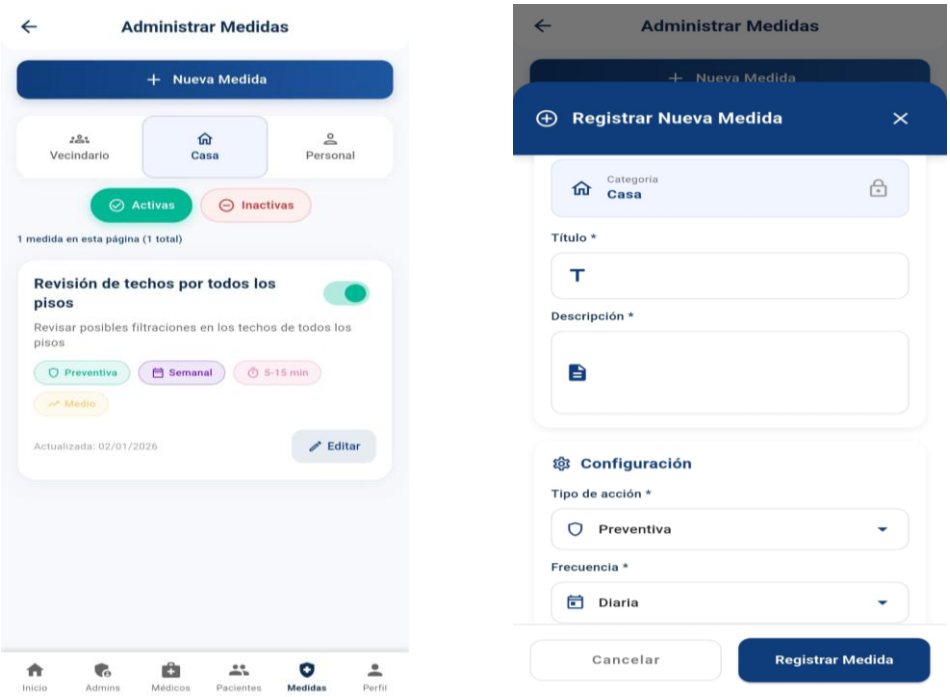





```python
def listar_pacientes_simplificado(request):
    """Versión básica con correo y último diagnóstico"""
    try:
        search_query = (request.GET.get("search") or "").strip()
        rol_paciente = Rol.objects.filter(nombre_rol__iexact="PACIENTE").first()

        if not rol_paciente:
            return Response(
                {"status": "error", "message": "No se encontró el rol PACIENTE."},
                status=status.HTTP_500_INTERNAL_SERVER_ERROR,
            )

        correo_subquery = Usuario.objects.filter(id_persona=OuterRef("id_persona")).values("correo")[:1]
        pacientes = (
            Persona.objects.filter(id_rol=rol_paciente.id_rol)
            .annotate(correo=Subquery(correo_subquery))
            .order_by("nombres", "apellidos")
        )

        if search_query:
            pacientes = pacientes.filter(
                Q(nombres__icontains=search_query)
                | Q(apellidos__icontains=search_query)
                | Q(dni__icontains=search_query)
            )

        serializer = PacienteListaSerializer(pacientes, many=True)

        return Response(
            {
                "status": "success",
                "total_pacientes": pacientes.count(),
                "rol_filtrado": rol_paciente.nombre_rol,
                "pacientes": serializer.data,
            },
            status=status.HTTP_200_OK,
        )

    except Exception as e:
        return Response(
            {"status": "error", "message": f"Error al obtener los pacientes: {str(e)}"},
            status=status.HTTP_500_INTERNAL_SERVER_ERROR,
        )
```

Descripción: se muestra el código para poder obtener la lista de pacientes totales registradas en el sistema. Se crea una función para recuperar los datos completos de los pacientes que se han registrado en la aplicación.

Pantalla de Medidas Preventivas




```python
# ====================================================
# CREAR MEDIDA (SOLO ADMINISTRADOR)
# ====================================================

@api_view(['POST'])
@permission_classes([TokenAuth])
def crear_medida(request):
    usuario = request.usuario

    if not _es_admin(usuario):
        return Response(
            {"status": "error", "message": "Solo los administradores pueden crear medidas."},
            status=status.HTTP_403_FORBIDDEN
        )

    data = request.data.copy()
    data['id_administrador'] = usuario.id_usuario
    data['created_at'] = timezone.now()
    data['updated_at'] = timezone.now()

    serializer = MedidaSerializer(data=data)

    if serializer.is_valid():
        with transaction.atomic():
            medida = serializer.save()
            return Response(
                {
                    "status": "success",
                    "message": "Medida creada correctamente.",
                    "data": MedidaSerializer(medida).data
                },
                status=status.HTTP_201_CREATED
            )

    return Response(
        {
            "status": "error",
            "message": "Datos inválidos.",
            "errors": serializer.errors
        },
        status=status.HTTP_400_BAD_REQUEST
    )
```

Descripción: se muestra el código para el botón **Registrar Medida.** Se crea una función para registrar los datos completos de una medida preventiva y por defecto se asigna como activa y dependiendo de en qué categoría nos encontremos ubicados en la aplicación.